

list core java topics need to know to 7-8 year experienced java developer without missing even single small topic

Absolutely. For a **7–8 year experienced Java developer**, you should know Core Java beyond syntax—you should be able to explain **internals, trade-offs, edge cases, concurrency, JVM behavior, and why Java behaves the way it does.**

Here is a **comprehensive Core Java checklist**, including small topics that are commonly skipped.

Core Java — 7–8 Year Experience Master Checklist

1. Java Fundamentals

Java platform

- What is Java?
- Features of Java
- Java history and versions
- Java SE vs Jakarta EE
- JDK
- JRE
- JVM
- JDK vs JRE vs JVM
- Java compilation process
- `.java` → `.class`
- Bytecode
- `javac`
- `java` command
- Classpath
- Module path
- Platform independence
- "Write once, run anywhere"
- Java source code execution flow

Important Java terminology

- Class
- Object

- Instance
 - Reference
 - Variable
 - Method
 - Field
 - Constructor
 - Package
 - Module
 - API
 - Library
 - Framework
-

2. Data Types & Variables

Primitive types

- byte
- short
- int
- long
- float
- double
- char
- boolean

Details

- Size of primitive types
- Range of primitive types
- Signed vs unsigned
- Integer overflow
- Floating-point precision
- IEEE 754 basics
- NaN
- Positive/negative infinity
- `-0.0`
- Character encoding basics
- Unicode
- UTF-8
- Escape sequences

Variables

- Local variables
 - Instance variables
 - Static variables
 - Parameters
 - Variable initialization
 - Default values
 - Scope
 - Lifetime
 - Shadowing
 - Variable hiding
-

3. Operators

- Arithmetic operators
 - Relational operators
 - Logical operators
 - Bitwise operators
 - Shift operators
 - Unary operators
 - Assignment operators
 - Compound assignment
 - Ternary operator
 - `instanceof`
 - Operator precedence
 - Operator associativity
 - Short-circuit evaluation
 - `&` vs `&&`
 - `|` vs `||`
 - `++` / `--`
 - Integer promotion
 - Type promotion
 - Overflow during operations
-

4. Control Flow

- `if`
- `else`

- Nested if
 - switch
 - Traditional switch
 - Switch expressions
 - case
 - default
 - yield
 - for
 - Enhanced for
 - while
 - do-while
 - break
 - continue
 - Labeled statements
 - Infinite loops
 - Reachability
 - Unreachable code
-

5. OOP — Extremely Important

Classes & Objects

- Class declaration
- Object creation
- Object references
- Object identity
- Object state
- Object behavior
- Instance members
- Static members

Encapsulation

- Getters/setters
- Immutable objects
- Defensive copying
- Information hiding

Inheritance

- extends

- Single inheritance
- Multilevel inheritance
- Hierarchical inheritance
- Why Java doesn't support multiple class inheritance
- Method inheritance
- Constructor inheritance misconception
- Field hiding

Polymorphism

- Compile-time polymorphism
- Method overloading
- Runtime polymorphism
- Method overriding
- Dynamic method dispatch
- Covariant return type
- Static method hiding
- Private method behavior
- Final method behavior

Abstraction

- Abstract classes
- Abstract methods
- Interfaces
- Interface default methods
- Interface static methods
- Interface private methods
- Functional interfaces
- Multiple interface inheritance

this and super

- `this`
- `this()`
- `super`
- `super()`
- `super.method()`
- `super.field`

6. Access Modifiers

- `public`
- `private`
- `protected`
- Default/package-private

Understand access:

- Same class
 - Same package
 - Different package subclass
 - Different package non-subclass
-

7. Constructors

- Default constructor
 - No-argument constructor
 - Parameterized constructor
 - Constructor overloading
 - Constructor chaining
 - `this()`
 - `super()`
 - Constructor execution order
 - Constructor and inheritance
 - Private constructors
 - Constructor accessibility
 - Constructor exceptions
 - Constructor cannot be overridden
 - Constructor cannot be inherited
-

8. `static`, `final`, `abstract`

`static`

- Static variable
- Static method
- Static block
- Static nested class
- Static initialization
- Static import
- Static method limitations

final

- Final variable
- Final reference
- Final method
- Final class
- Blank final variable
- Final parameter
- Immutability vs final

abstract

- Abstract class
- Abstract method
- Abstract class constructor
- Abstract class with concrete methods
- Abstract class vs interface

9. Object Class

Know every important method:

 Java



- toString()
- equals()
- hashCode()
- getClass()
- clone()
- finalize() — historical/deprecated concept
- wait()
- notify()
- notifyAll()

Especially:

- equals() contract
- hashCode() contract
- Why overriding equals() requires hashCode()
- Identity vs equality
- == VS equals()

10. String — Very Important

- String immutability
- String pool
- String literals
- `new String()`
- String interning
- `intern()`
- `==` VS `equals()`
- String concatenation
- Compile-time concatenation
- Runtime concatenation
- `StringBuilder`
- `StringBuffer`
- String performance
- String memory
- UTF-16 representation basics
- Important String methods
- `substring()`
- `charAt()`
- `indexOf()`
- `contains()`
- `split()`
- `replace()`
- `replaceAll()`
- `startsWith()`
- `endsWith()`
- `trim()`
- `strip()`
- `isBlank()`
- `isEmpty()`
- `join()`
- `formatted()`

11. Wrapper Classes & Autoboxing

- Integer

- Long
- Double
- Float
- Short
- Byte
- Character
- Boolean

Understand:

- Boxing
 - Unboxing
 - Autoboxing
 - Auto-unboxing
 - Wrapper immutability
 - Integer cache
 - Wrapper `==`
 - Wrapper `equals()`
 - `NullPointerException` during unboxing
 - Primitive vs wrapper performance
 - `valueOf()`
 - `parseInt()`
 - `xxxValue()`
-

12. Type Casting

- Widening conversion
 - Narrowing conversion
 - Primitive casting
 - Reference casting
 - Upcasting
 - Downcasting
 - Explicit casting
 - Implicit casting
 - `ClassCastException`
 - `instanceof`
 - Pattern matching with `instanceof`
-

13. Packages & Imports

- Package declaration
 - Import
 - Static import
 - Wildcard import
 - Fully qualified class name
 - Package-private access
 - Package naming conventions
 - Classpath
 - Package vs module
-

14. Arrays

- One-dimensional arrays
 - Multidimensional arrays
 - Jagged arrays
 - Array initialization
 - Anonymous arrays
 - Array covariance
 - Primitive arrays
 - Object arrays
 - `ArrayIndexOutOfBoundsException`
 - `ArrayStoreException`
 - `Arrays` utility class
 - `Arrays.sort()`
 - `Arrays.binarySearch()`
 - `Arrays.copyOf()`
 - `Arrays.equals()`
 - `Arrays.deepEquals()`
 - `Arrays.asList()`
 - `Array` → `List`
 - `List` → `Array`
 - `System.arraycopy()`
-

15. Exception Handling — Very Important

Basics

- Exception
- Error
- Throwable
- Checked exception
- Unchecked exception
- RuntimeException
- Custom exceptions

Keywords

- try
- catch
- finally
- throw
- throws

Advanced

- Multiple catch
- Multi-catch
- Nested try
- Try-with-resources
- Suppressed exceptions
- Exception chaining
- Cause
- `initCause()`
- `getCause()`
- Stack trace
- Exception propagation
- Rethrowing exceptions
- Precise rethrow
- Custom checked exception
- Custom unchecked exception

Important questions

- Can finally override return?
- What happens if finally throws exception?
- Can try have only finally?
- Can catch exist without try?
- Checked vs unchecked
- Exception vs Error
- throw VS throws

16. Generics — Very Important for Experienced Developers

- Generic classes
 - Generic methods
 - Generic interfaces
 - Generic constructors
 - Type parameters
 - Multiple type parameters
 - Bounded types
 - Upper bounds
 - Lower bounds
 - Wildcards
 - `<?>`
 - `<? extends T>`
 - `<? super T>`
 - PECS
 - Generic inheritance
 - Raw types
 - Type erasure
 - Heap pollution
 - Generic arrays
 - Generic varargs
 - `@SafeVarargs`
 - Restrictions on generics
 - Why `List<Integer>` isn't `List<Number>`
 - Generic method type inference
 - Diamond operator
 - Recursive generics
-

17. Collections Framework — Must Master

Interfaces

- `Collection`
- `List`
- `Set`
- `Queue`
- `Deque`

- Map

List

- ArrayList
- LinkedList
- Vector
- Stack
- CopyOnWriteArrayList

Set

- HashSet
- LinkedHashSet
- TreeSet
- EnumSet
- CopyOnWriteArraySet

Map

- HashMap
- LinkedHashMap
- TreeMap
- Hashtable
- ConcurrentHashMap
- WeakHashMap
- IdentityHashMap
- EnumMap

Queue/Deque

- PriorityQueue
- ArrayDeque
- Blocking queues

Must understand internally

- ArrayList resizing
- LinkedList node structure
- HashMap hashing
- HashMap bucket
- Collision
- HashMap treeification
- Load factor

- Capacity
- Threshold
- Rehashing
- HashMap Java 7 vs Java 8+
- HashSet internally using HashMap
- LinkedHashMap ordering
- TreeMap Red-Black Tree
- TreeSet internally
- PriorityQueue heap
- ConcurrentHashMap internals
- Fail-fast iterator
- Fail-safe terminology
- Weakly consistent iteration
- `modCount`

Collection contracts

- `equals()`
 - `hashCode()`
 - `Comparable`
 - `Comparator`
-

18. Comparable & Comparator

- `Comparable`
 - `Comparator`
 - Natural ordering
 - Custom ordering
 - `compareTo()`
 - `compare()`
 - Comparator chaining
 - `thenComparing()`
 - `reversed()`
 - Null handling
 - Consistency with `equals`
-

19. Functional Programming

- Functional interfaces

- `@FunctionalInterface`
- Lambda expressions
- Lambda syntax
- Target typing
- Lambda variable capture
- Effectively final variables
- Method references
- Constructor references
- Built-in functional interfaces

Important interfaces

- `Predicate`
 - `Consumer`
 - `Supplier`
 - `Function`
 - `BiFunction`
 - `UnaryOperator`
 - `BinaryOperator`
 - `BiPredicate`
 - `BiConsumer`
-

20. Stream API — Very Important

- Stream creation
- Intermediate operations
- Terminal operations
- Lazy evaluation
- Pipeline
- `filter`
- `map`
- `flatMap`
- `mapToInt`
- `mapToLong`
- `mapToDouble`
- `distinct`
- `sorted`
- `limit`
- `skip`

- peek
- takeWhile
- dropWhile

Terminal operations

- forEach
- forEachOrdered
- collect
- reduce
- count
- min
- max
- findFirst
- findAny
- anyMatch
- allMatch
- noneMatch

Collectors

- toList
- toSet
- toMap
- groupingBy
- partitioningBy
- mapping
- joining
- counting
- summarizing
- reducing
- Downstream collectors

Advanced

- Sequential vs parallel streams
- Parallel stream internals
- ForkJoinPool
- Stream ordering
- Stateful vs stateless operations
- Short-circuiting

- Side effects
 - Stream reuse
 - Spliterator
 - Custom collectors
-

21. Date & Time API

- LocalDate
- LocalTime
- LocalDateTime
- ZonedDateTime
- OffsetDateTime
- OffsetTime
- Instant
- Duration
- Period
- ZoneId
- ZoneOffset
- DateTimeFormatter
- Temporal
- TemporalAdjuster
- Legacy Date
- Calendar
- Time zones
- UTC
- Epoch
- DST
- Formatting
- Parsing
- Date arithmetic

[✦ Upgrade](#)

22. Optional

- Why Optional exists
- Creating Optional
- of()
- ofNullable()

- `empty()`
 - `isPresent()`
 - `isEmpty()`
 - `ifPresent()`
 - `ifPresentOrElse()`
 - `orElse()`
 - `orElseGet()`
 - `orElseThrow()`
 - `map()`
 - `flatMap()`
 - `filter()`
 - `or()`
 - Optional chaining
 - Optional anti-patterns
 - Optional as method return type
 - Optional fields/parameters debate
-

23. Multithreading — Critical for 7–8 Years

Thread basics

- Process vs thread
- Thread lifecycle
- Thread states
- Creating threads
- Thread
- Runnable
- Callable
- Future

Thread methods

- `start()`
- `run()`
- `sleep()`
- `join()`
- `interrupt()`
- `isInterrupted()`
- `interrupted()`

- `yield()`
- `setDaemon()`
- `isAlive()`

Synchronization

- Race condition
- Critical section
- `synchronized`
- Synchronized method
- Synchronized block
- Object monitor
- Intrinsic lock
- Class-level lock
- Instance-level lock

Coordination

- `wait()`
 - `notify()`
 - `notifyAll()`
 - Producer-consumer
 - Spurious wakeups
 - `wait()` must be called while owning monitor
 - Why wait belongs inside synchronized
-

24. Java Memory Model — Very Important

You should understand:

- Heap
- Stack
- Method area/metaspace
- Thread-local memory
- Shared memory
- Visibility
- Atomicity
- Ordering
- Happens-before
- Data race
- Race condition
- Instruction reordering

- Memory barriers
 - `volatile`
 - `final` field visibility
 - Safe publication
 - Unsafe publication
 - Double-checked locking
 - Immutability and thread safety
-

25. Locks & Concurrency Utilities

Locks

- `Lock`
- `ReentrantLock`
- `ReadWriteLock`
- `ReentrantReadWriteLock`
- `StampedLock`

Synchronizers

- `CountDownLatch`
- `CyclicBarrier`
- `Semaphore`
- `Phaser`
- `Exchanger`

Atomic classes

- `AtomicInteger`
 - `AtomicLong`
 - `AtomicBoolean`
 - `AtomicReference`
 - Atomic arrays
 - CAS
 - Compare-and-swap
 - Lock-free programming
 - ABA problem
-

26. Executor Framework

- Executor
- ExecutorService
- ScheduledExecutorService
- ThreadPoolExecutor
- ScheduledThreadPoolExecutor
- Future
- Callable
- Runnable
- FutureTask

Thread pool concepts

- Core pool size
- Maximum pool size
- Queue
- Keep-alive time
- Thread factory
- Rejected execution handler
- Work queue
- Shutdown
- shutdown()
- shutdownNow()
- awaitTermination()

Rejection policies

- AbortPolicy
- CallerRunsPolicy
- DiscardPolicy
- DiscardOldestPolicy

27. CompletableFuture

- CompletableFuture
- Async programming
- supplyAsync
- runAsync
- thenApply
- thenAccept
- thenRun

- `thenCompose`
 - `thenCombine`
 - `allOf`
 - `anyOf`
 - `exceptionally`
 - `handle`
 - `whenComplete`
 - Async vs non-async methods
 - Executor selection
 - Exception propagation
 - Combining asynchronous tasks
-

28. Deadlock & Concurrency Problems

Know:

- Deadlock
- Livelock
- Starvation
- Race condition
- Data race
- Thread contention
- Lock contention
- Priority inversion

Deadlock prevention

- Lock ordering
 - Timeout
 - `tryLock()`
 - Reducing lock scope
 - Avoid nested locks
-

29. JVM Internals — Must Know at 7–8 Years

JVM architecture

- Class Loader
- Runtime data areas
- Execution Engine

- JIT compiler
- Garbage Collector
- Native method interface

Class loading

- Loading
- Linking
- Verification
- Preparation
- Resolution
- Initialization

Class loaders

- Bootstrap
 - Platform
 - Application
 - Custom class loader
 - Parent delegation model
 - ClassLoader hierarchy
 - Class loading conflicts
-

30. JVM Memory

- Heap
 - Young generation
 - Eden
 - Survivor spaces
 - Old generation
 - Metaspace
 - Code cache
 - Thread stack
 - PC register
 - Native method stack
 - Direct memory
 - Off-heap memory
-

31. Garbage Collection

Understand conceptually and practically:

- Garbage
- Reachability
- GC roots
- Mark
- Sweep
- Compact
- Generational GC
- Minor GC
- Major GC
- Full GC
- Stop-the-world
- Safepoint

Collectors

- Serial GC
- Parallel GC
- G1 GC
- ZGC
- Shenandoah
- CMS — historical knowledge

Important

- Memory leak in Java
- `OutOfMemoryError`
- `StackOverflowError`
- GC tuning basics
- Heap dump
- GC logs

32. JIT & JVM Execution

- Interpreter
- JIT compiler
- HotSpot
- Method inlining
- Escape analysis
- Dead-code elimination
- Runtime optimization
- Tiered compilation

- Deoptimization
 - Native code
-

33. Reflection

- Reflection API
 - Class
 - Method
 - Field
 - Constructor
 - Dynamic method invocation
 - Accessing private members
 - `setAccessible`
 - Reflection performance
 - Reflection security concerns
 - Annotation processing through reflection
-

34. Annotations

Built-in

- `@Override`
- `@Deprecated`
- `@SuppressWarnings`
- `@FunctionalInterface`
- `@SafeVarargs`

Custom annotations

- Annotation declaration
- Meta-annotations
- `@Target`
- `@Retention`
- `@Documented`
- `@Inherited`
- `@Repeatable`

Retention

- `SOURCE`

- CLASS
 - RUNTIME
-

35. Enums

- Enum declaration
 - Enum fields
 - Enum constructors
 - Enum methods
 - Enum-specific behavior
 - Enum inheritance restrictions
 - `values()`
 - `valueOf()`
 - `ordinal()`
 - EnumMap
 - EnumSet
 - Enum as singleton
 - Enum thread safety
 - Enum initialization
-

36. Inner & Nested Classes

- Member inner class
 - Static nested class
 - Local inner class
 - Anonymous inner class
 - Outer class reference
 - Variable capture
 - Effectively final variables
 - Inner class bytecode concept
-

37. Interfaces — Advanced

- Default methods
- Static methods
- Private methods
- Diamond problem

- Default method conflict
 - Interface inheritance
 - Functional interfaces
 - Marker interfaces
 - Multiple interface implementation
-

38. Immutability

Know how to design immutable classes:

- `final` class
 - Private final fields
 - No setters
 - Constructor initialization
 - Defensive copying
 - Defensive copying on getters
 - Mutable field problem
 - Deep vs shallow immutability
 - Thread safety through immutability
 - Records and immutability limitations
-

39. Records

For modern Java:

- Record declaration
 - Canonical constructor
 - Compact constructor
 - Accessors
 - `equals`
 - `hashCode`
 - `toString`
 - Record limitations
 - Mutable components
 - Record vs POJO
 - Record vs Lombok
-

40. Sealed Classes

- sealed
 - permits
 - non-sealed
 - final
 - Sealed interfaces
 - Restricted inheritance
 - Pattern matching interaction
-

41. Modern Java Features

Depending on the Java version used by your company, know the major evolution:

- Java 8 — Lambdas, Streams, Optional, Date/Time
- Java 9 — Modules
- Java 10 — var
- Java 11 — APIs and HTTP Client
- Java 14+ — Switch expressions
- Java 15+ — Text blocks
- Java 16+ — Records
- Java 17 — Sealed classes
- Java 21 — Pattern matching, virtual threads, sequenced collections
- Modern Java pattern matching
- Virtual threads
- Structured concurrency concepts
- Scoped values concepts

For a current 7–8 year developer, **Java 8 + Java 17 + modern Java 21 concepts** are particularly valuable.

42. Java Modules

- Module
- module-info.java
- requires
- exports
- opens
- uses
- provides
- requires transitive

- Named modules
 - Automatic modules
 - Unnamed module
 - Module path vs classpath
-

43. I/O

Classic I/O

- InputStream
- OutputStream
- Reader
- Writer
- File I/O
- Buffered streams
- Byte streams
- Character streams

NIO

- Path
 - Paths
 - Files
 - Channel
 - Buffer
 - ByteBuffer
 - Selector
 - Non-blocking I/O
 - File operations
 - Directory operations
 - File attributes
-

44. Serialization

- Serializable
- Object serialization
- Deserialization
- serialVersionUID
- transient

- Externalizable
 - Serialization lifecycle
 - Custom serialization
 - readObject
 - writeObject
 - Serialization vulnerabilities
 - Why Java serialization is generally avoided in modern systems
-

45. Networking

Core Java knowledge:

- Socket
 - ServerSocket
 - TCP
 - UDP
 - DatagramSocket
 - InetAddress
 - URL
 - URI
 - HTTP client
 - Blocking vs non-blocking networking
-

46. Regular Expressions

- Regex basics
 - Pattern
 - Matcher
 - Character classes
 - Quantifiers
 - Groups
 - Capturing groups
 - Non-capturing groups
 - Lookahead
 - Lookbehind
 - Greedy vs reluctant quantifiers
-

47. JDBC — Core Java Adjacent

For senior Java interviews, know:

- JDBC architecture
 - Driver
 - Connection
 - Statement
 - PreparedStatement
 - CallableStatement
 - ResultSet
 - Transactions
 - Commit
 - Rollback
 - Batch operations
 - Connection pooling
 - SQL injection
 - Resource management
 - Try-with-resources
-

48. Internationalization

- Locale
 - ResourceBundle
 - Currency
 - Number formatting
 - Date formatting
 - Unicode
 - Character encoding
-

49. Security Basics in Core Java

- Java cryptography basics
- MessageDigest
- Hashing vs encryption
- SecureRandom
- Cipher
- Key
- Digital signatures

- KeyStore
- TrustStore
- SSL/TLS concepts
- Secure coding
- Serialization security

You don't need to become a security specialist, but you should understand the fundamentals.

50. Design Patterns in Java

At 7–8 years, know how these are implemented **using Core Java**:

Creational

- Singleton
- Factory
- Abstract Factory
- Builder
- Prototype

Structural

- Adapter
- Decorator
- Proxy
- Facade
- Composite
- Bridge

Behavioral

- Strategy
- Observer
- Template Method
- Chain of Responsibility
- Command
- State
- Iterator
- Mediator

Especially:

- Singleton thread safety
- Double-checked locking

- Enum singleton
 - Factory vs Abstract Factory
 - Strategy vs State
 - Proxy vs Decorator
-

51. Java Performance

- Big-O basics
 - Object creation cost
 - Boxing/unboxing overhead
 - String concatenation
 - Collection selection
 - ArrayList vs LinkedList
 - HashMap performance
 - Stream overhead
 - Parallel streams
 - Thread creation cost
 - Thread pools
 - Lock contention
 - GC overhead
 - Memory allocation
 - Escape analysis
 - JIT optimizations
-

52. Java Debugging & Troubleshooting

A 7–8 year developer should be comfortable investigating:

- High CPU
- High memory
- Memory leak
- Thread deadlock
- Thread starvation
- Slow application
- GC pauses
- `OutOfMemoryError`
- `StackOverflowError`
- `ConcurrentModificationException`
- `ClassNotFoundException`

- `NoClassDefFoundError`
- `ClassCastException`
- `NullPointerException`

Tools/concepts:

- Thread dump
- Heap dump
- GC logs
- JFR
- JMC
- `jstack`
- `jmap`
- `jcmd`
- VisualVM
- JVM options

53. Important Java "Tricky" Areas

You should be able to solve/explain questions involving:

- `==` VS `equals()`
- String pool
- Integer cache
- `final` reference
- Static initialization
- Initialization order
- Parent/child constructor execution
- Method overloading resolution
- Method overriding
- Covariant returns
- Exception rules during overriding
- Static method hiding
- Private methods
- Default interface methods
- Generic type erasure
- Type casting
- Autoboxing
- Null unboxing
- HashMap collisions
- Mutable HashMap keys

- ConcurrentModificationException
 - Fail-fast iterators
 - volatile
 - synchronized
 - wait() / notify()
 - Deadlocks
 - Thread pools
 - CompletableFuture
 - GC
 - Class loading
-

54. Java Coding Skills

For 7–8 years, don't only study theory. You should be able to implement:

- Immutable class
 - Thread-safe Singleton
 - LRU cache
 - Custom HashMap
 - Custom ArrayList
 - Producer-consumer
 - Thread-safe counter
 - Rate limiter
 - Custom Comparator
 - Custom Collector
 - Custom annotation
 - Reflection-based utility
 - Thread pool example
 - CompletableFuture workflow
 - Deadlock example/fix
 - Blocking queue
 - Singleton using enum
 - Deep copy
 - Clone alternatives
-

55. Core Java Interview Depth

For every major topic, aim for 4 levels:

Level 1 — What?

Example:

What is HashMap?

Level 2 — How?

How does HashMap work internally?

Level 3 — Why?

Why does HashMap use treeification?

Level 4 — Production/trade-off

When would you choose HashMap vs ConcurrentHashMap, and what are the concurrency implications?

For a **7–8 year interview**, Level 3 and Level 4 are what separate you from a 2–4 year candidate.

Priority for Your Preparation

Given your **Java + Spring Boot backend** target, I would prioritize the massive list above like this:

Tier 1 — Must be extremely strong

1. OOP
2. String
3. Object>equals/hashCode
4. Collections
5. Generics
6. Exception handling
7. Multithreading
8. Java Memory Model
9. Synchronization
10. Executor Framework
11. CompletableFuture
12. Streams
13. JVM internals
14. Garbage Collection

15. Class loading

● Tier 2 — Strong knowledge

16. Functional interfaces/Lambda

17. Optional

18. Comparable/Comparator

19. Immutable classes

20. Reflection

21. Annotations

22. Enums

23. Date/Time API

24. I/O/NIO

25. Serialization

26. Records

27. Sealed classes

28. Modern Java features

● Tier 3 — Know the concepts

29. Modules

30. Networking

31. Regex

32. Internationalization

33. JDBC

34. Java security basics

35. Design patterns

36. JVM debugging tools

Important: Don't try to memorize this as one giant list. For your interview preparation, I'd recommend going **topic-by-topic from fundamentals → internals → interview traps → coding examples**. That will make this manageable rather than turning it into a checklist you forget after a few days.



ChatGPT can make mistakes. Check important info.